



Unidad 4

Layout managers: ubicar componentes sin coordenadas

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026



La vista define la disposición relativa. Las coordenadas en píxeles las calcula un objeto aparte.

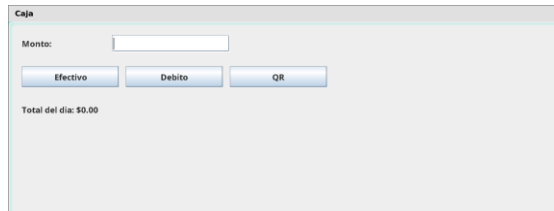
Ese objeto es el layout manager. El contenedor depende de la interfaz, como Caja de Notificador.

Sin layout manager

```
ventana.setLayout(null);

etiquetaMonto.setBounds(20, 20, 120, 25);
campoMonto.setBounds(150, 20, 170, 25);
botonEfectivo.setBounds(20, 65, 140, 30);
botonDebito.setBounds(170, 65, 140, 30);
botonQR.setBounds(320, 65, 140, 30);
etiquetaTotal.setBounds(20, 115, 300, 25);

// Coordinadas fijas: al agrandar la ventana
// nada se reubica, y un boton nuevo obliga
// a recalcular las de los demas.
```



Un cambio visual implica recalcular coordenadas. Al agrandar, nada se reubica.



Layout manager

- Todo contenedor (**JFrame**, **JPanel**) tiene uno: un objeto que implementa **LayoutManager** y calcula posición y tamaño de cada hijo.
- El contenedor lo invoca al mostrarse, al cambiar de tamaño y al agregar un componente. **setLayout()** elige cuál.
- Usa el tamaño preferido de cada componente, **getPreferredSize()**: en un botón, el texto; en un campo, sus columnas, **new JTextField(12)**.

El layout manager calcula las coordenadas cada vez que el contenedor cambia. La vista no las escribe.

FlowLayout

```
JPanel panel = new JPanel();    // trae FlowLayout
panel.add(new JButton("Efectivo"));
panel.add(new JButton("Debito"));
panel.add(new JButton("QR"));
panel.add(new JButton("Transferencia"));
panel.add(new JButton("Cuenta corriente"));

// Alineacion y separacion, en pixeles:
// new FlowLayout(FlowLayout.LEFT, 8, 8)

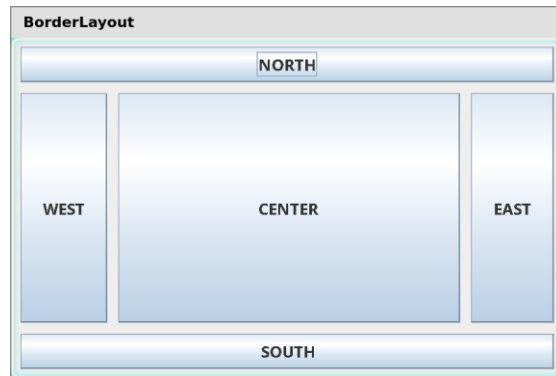
// Dos anchos de la misma ventana.
```



FlowLayout: en fila, en el orden del `add()`, cada uno con su tamaño preferido. Lo que no entra pasa a la fila siguiente.

BorderLayout

```
// El que trae un JFrame. El segundo  
// parametro de add() es la region.  
ventana.setLayout(new BorderLayout(8, 8));  
ventana.add(botonNorte, BorderLayout.NORTH);  
ventana.add(botonSur, BorderLayout.SOUTH);  
ventana.add(botonOeste, BorderLayout.WEST);  
ventana.add(botonEste, BorderLayout.EAST);  
ventana.add(botonCentro, BorderLayout.CENTER);  
  
// Norte y sur: todo el ancho, su altura.  
// Este y oeste: su ancho, la altura restante.  
// Centro: todo el espacio restante.
```



BorderLayout: cinco regiones, un componente por región. Un segundo add () en la misma región reemplaza al primero.

GridLayout

```
// filas, columnas, separacion horizontal, vertical
JPanel panel = new JPanel(new GridLayout(2, 3, 8, 8));
panel.add(new JButton("Efectivo"));
panel.add(new JButton("Debito"));
panel.add(new JButton("QR"));
panel.add(new JButton("Transferencia"));
panel.add(new JButton("Cuenta corriente"));
panel.add(new JButton("Anular"));

// Un 0 es "las que hagan falta":
// new GridLayout(1, 0) es una fila.
```



GridLayout: celdas iguales, una por componente, en el orden del `add()`. Las celdas ocupan todo el panel.

Paneles anidados

```
// Un JPanel es un componente (va en una
// region) y un contenedor (tiene su layout).
JPanel panelMonto = new JPanel(new FlowLayout());
panelMonto.add(etiquetaMonto);
panelMonto.add(campoMonto);
JPanel panelBotones = new JPanel(new GridLayout(1, 0));
panelBotones.add(botonEfectivo);
panelBotones.add(botonDebito);
panelBotones.add(botonQR);
ventana.add(panelMonto, BorderLayout.NORTH);
ventana.add(panelBotones, BorderLayout.CENTER);
ventana.add(etiquetaTotal, BorderLayout.SOUTH);
ventana.pack(); // en vez de setSize()
```

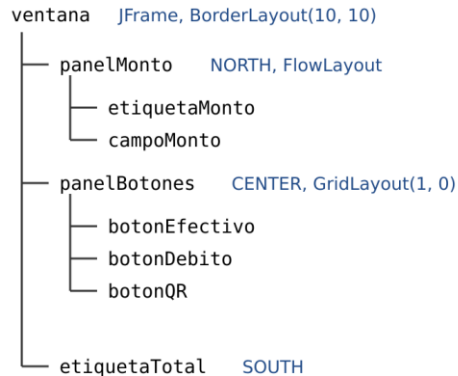
The screenshot shows a Java Swing window titled "Caja". Inside the window, there is a nested panel structure. At the top, there is a label "Monto:" followed by a text input field containing the value "1500". Below this, there are three buttons labeled "Efectivo", "Debito", and "QR" arranged horizontally. At the bottom of the window, there is a label "Total del dia: \$0.00". The window's layout demonstrates the use of nested panels and BorderLayout.

Una región, un panel. Adentro del panel, su propio layout. `pack()` dimensiona la ventana según sus componentes.

Quién contiene a quién



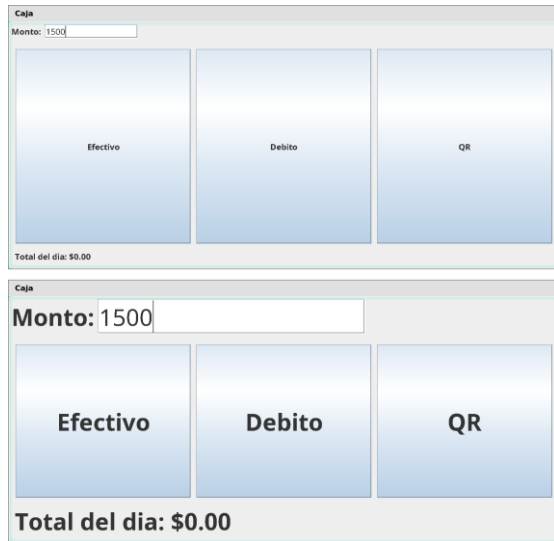
Quién contiene a quién



Cada layout ubica solo a sus hijos directos. Agregar el QR: crear el botón y un `add()`.

El tamaño también cuenta

```
// Al agrandar, la fuente queda en 12 puntos (arriba).  
// Pantalla táctil: la fuente se escala (abajo):  
ventana.addComponentListener(new ComponentAdapter() {  
    public void componentResized(ComponentEvent e) {  
        escalarFuentes();  
    }  
});  
private void escalarFuentes() {  
    float factor = (float) ventana.getHeight()  
                  / altoInicial;  
    escalar(getContentPane(), fuenteInicial * factor);  
    ventana.revalidate(); // los layouts recalculan  
}
```



El layout ubica y dimensiona componentes. El tamaño del texto se decide aparte, y a veces la ubicación sola no alcanza.



Otros layouts

- **BoxLayout**: una sola fila o una sola columna, sin pasar a la siguiente.
- **GridBagLayout**: grilla con celdas de distinto tamaño; un componente puede ocupar varias. El más flexible y el más largo de escribir.
- **CardLayout**: un componente visible por vez, y se cambia por código.
- Con **BorderLayout**, **FlowLayout**, **GridLayout** y paneles anidados sale casi cualquier ventana de escritorio.

Cambiar la disposición de un panel es cambiar un objeto. Los `add()` no cambian.



Resumen

- **setLayout (null)** y **setBounds ()** : coordenadas fijas. Al agrandar, nada se reubica.
- Un layout manager implementa **LayoutManager** y calcula las coordenadas cada vez que el contenedor cambia.
- **FlowLayout**: en fila, tamaño preferido. **BorderLayout**: cinco regiones. **GridLayout**: celdas iguales.
- Una región, un panel, con su propio layout. **pack ()** en vez de **setSize ()**.
- El controlador se registra en los mismos métodos: solo cambió la vista.



La actividad de hoy

- El conversor de la clase pasada, con tres botones y coordenadas a mano. La ventana ya se puede agrandar.
- **Parte 1:** agrandar la ventana y ver que nada se reubica. Un texto más largo se recorta.
- **Parte 2:** **BorderLayout** en la ventana. La etiqueta y el campo en un panel al norte, el resultado al sur, los botones en un panel al centro. **pack ()** . Probar **FlowLayout** contra **GridLayout**.
- **Parte 3:** las libras. Crear el botón y un **add ()** : ninguna coordenada que recalcular.